1. Project Abstract

The proliferation of Internet Protocol (IP) cameras has generated an unprecedented volume of video data, creating a critical need for automated, intelligent analysis. Traditional video management systems often function as passive recording tools, relying heavily on human operators for monitoring and threat detection, which is inefficient, error-prone, and unscalable. This project presents the design and implementation of a Multi-Agent Video Analysis System, a web-based platform engineered to transform passive surveillance into an active, intelligent monitoring solution. The system ingests real-time video streams from multiple IP cameras via RTSP/HTTP protocols and orchestrates a suite of specialized AI agents for concurrent analysis. These agents, including YOLOv8 for object detection, EasyOCR for license plate recognition, and OpenCV-based modules for face detection and motion tracking, process video frames to extract meaningful events and metadata. A key innovation is the use of WebRTC to stream the processed video with overlaid analytical data to a dynamic React-based dashboard, ensuring ultra-low latency. The platform also ensures data integrity by simultaneously recording the raw video to local storage. Built on a robust technology stack featuring a FastAPI backend and containerized with Docker for scalable deployment, this system provides a comprehensive, efficient, and accessible solution for modern security and monitoring challenges.

2. Problem Statement

Conventional video surveillance systems are primarily reactive, serving as little more than digital recorders for post-incident forensic analysis. The reliance on human operators to continuously monitor multiple feeds is cognitively demanding, leading to fatigue and a high probability of missing critical events. Furthermore, the vast amount of unanalyzed video data makes it impossible to perform efficient, real-time searches for specific objects, persons, or activities. While some systems offer basic motion detection, they often generate excessive false alarms and lack the intelligence to differentiate between trivial movements and genuine security threats. There is a significant technological gap between simple video recording and the need for proactive, intelligent, and real-time situational awareness. This project addresses this gap by developing a system that automates the analysis process, providing actionable intelligence instantly rather than just storing raw footage.

3. Objectives

The primary goal of this project is to develop a scalable and intelligent video analysis platform.

1. To design and develop a multi-agent architecture for concurrent video analysis.
   Explanation: This involves creating a modular system where independent AI agents (for object detection, LPR, etc.) can process video streams in parallel. This design enhances performance, allows for scalability, and makes the system easily extensible with new analytical capabilities.

2. To implement real-time object detection and tracking using YOLOv8.
   Explanation: The system will utilize the state-of-the-art YOLOv8 model to identify and classify objects of interest (e.g., persons, vehicles) within the video feed and assign them unique IDs to track their movement across frames, enabling behavior analysis.

3. To integrate Optical Character Recognition for License Plate Recognition (LPR) using EasyOCR.
   Explanation: This objective focuses on enabling the system to automatically read and log license plate numbers from detected vehicles, a crucial feature for traffic monitoring, access control, and law enforcement applications.

4. To enable low-latency video streaming to a web client using WebRTC.
   Explanation: The project will implement WebRTC to stream the AI-processed video directly to a browser-based dashboard. This is essential for providing users with immediate, real-t ime visual feedback without the delays associated with traditional streaming protocols like HLS or RTMP.

5. To create an interactive and user-friendly dashboard using React.js.
   Explanation: The frontend will be a dynamic web application built with React, allowing users to view multiple live streams, monitor analytical alerts, and manage system settings, providing a centralized and intuitive control interface.

6. To ensure reliable video archiving by storing raw video streams to local storage.
   Explanation: While analysis is key, the ability to review the original, unaltered footage is critical. This objective ensures that a continuous recording of the raw video feed is saved for evidentiary or review purposes.

4. Scope and Limitations

Scope

The project encompasses the development of a complete, end-to-end web-based video analysis system.

The scope includes:

1. Connecting to and ingesting video from multiple IP cameras using standard protocols (RTSP, HTTP).

2.Implementing a multi-agent AI pipeline for object detection, license plate recognition, face detection, and motion tracking.

3.Developing a real-time streaming module using WebRTC to deliver processed video to a web client.

Limitations

Computational Resources: The performance of the AI models is directly dependent on the hardware (CPU/GPU) available. The number of simultaneous streams and the complexity of analysis may be limited by the host machine's capabilities.

1. Network Dependency: The system's performance, particularly for real-time streaming and remote camera access, is highly dependent on network bandwidth and stability.
2. Model Accuracy: The accuracy of the AI agents is bound by the training of the pre-trained models (YOLOv8, EasyOCR). Performance may degrade in poor lighting, adverse weather conditions, or with low-resolution video feeds.
3. Database Integration: The current scope focuses on real-time analysis and local video storage. Long-term storage and querying of analytical metadata (e.g., logged license plates) in a dedicated database are outside the primary scope.

5. Target Audience and Real-World Applications

Target Audience

1. Final Year Engineering Students: As a comprehensive project demonstrating full-stack development, AI/ML integration, and system architecture.
2. Security Professionals and System Integrators: Looking for an open, customizable alternative to expensive commercial Video Management Software (VMS).
3. Small to Medium-sized Enterprises (SMEs): Who need affordable, intelligent surveillance for their premises.
4. Researchers and Developers: Interested in a platform for prototyping and deploying multi-agent video analytics systems.

Real-World Applications

Urban Traffic Management and Control

Real-World Applications:

1. Smart City & Traffic Management: Monitoring traffic flow, detecting violations (e.g., red-light running), and managing parking systems through automated license plate recognition.

2. Perimeter Security: Automatically detecting and alerting on intrusions in restricted areas for campuses, industrial sites, or residential communities.

3. Retail Analytics: Analyzing customer footfall patterns, queue lengths, and dwell times to optimize store layout and staffing.

4. Access Control: Automating entry gates for authorized vehicles or personnel by integrating LPR and face detection.

5. Industrial Safety: Enforcing safety protocols by detecting if personnel are wearing required safety gear (e.g., helmets, vests) in hazardous zones.

Detected violations including overspeeding and rule-breaking trigger immediate alerts to traffic control centres for appropriate action.

Smart City Surveillance Infrastructure

As part of broader smart city initiatives, the platform can integrate with existing camera networks to provide intelligent monitoring across metropolitan areas. The multi-agent architecture supports city-wide scalability, enabling continuous traffic intelligence collection.

Automated Violation Enforcement

License plate recognition capabilities enable automated identification of violating vehicles for enforcement purposes. The system can log plate numbers, violation types, and timestamps, generating evidence for citation issuance.

Emergency Vehicle Prioritisation

Motion tracking and detection agents can identify emergency vehicles and automatically adjust traffic signal timing to create green corridors, reducing emergency response times and improving public safety.

Traffic Analytics and Planning

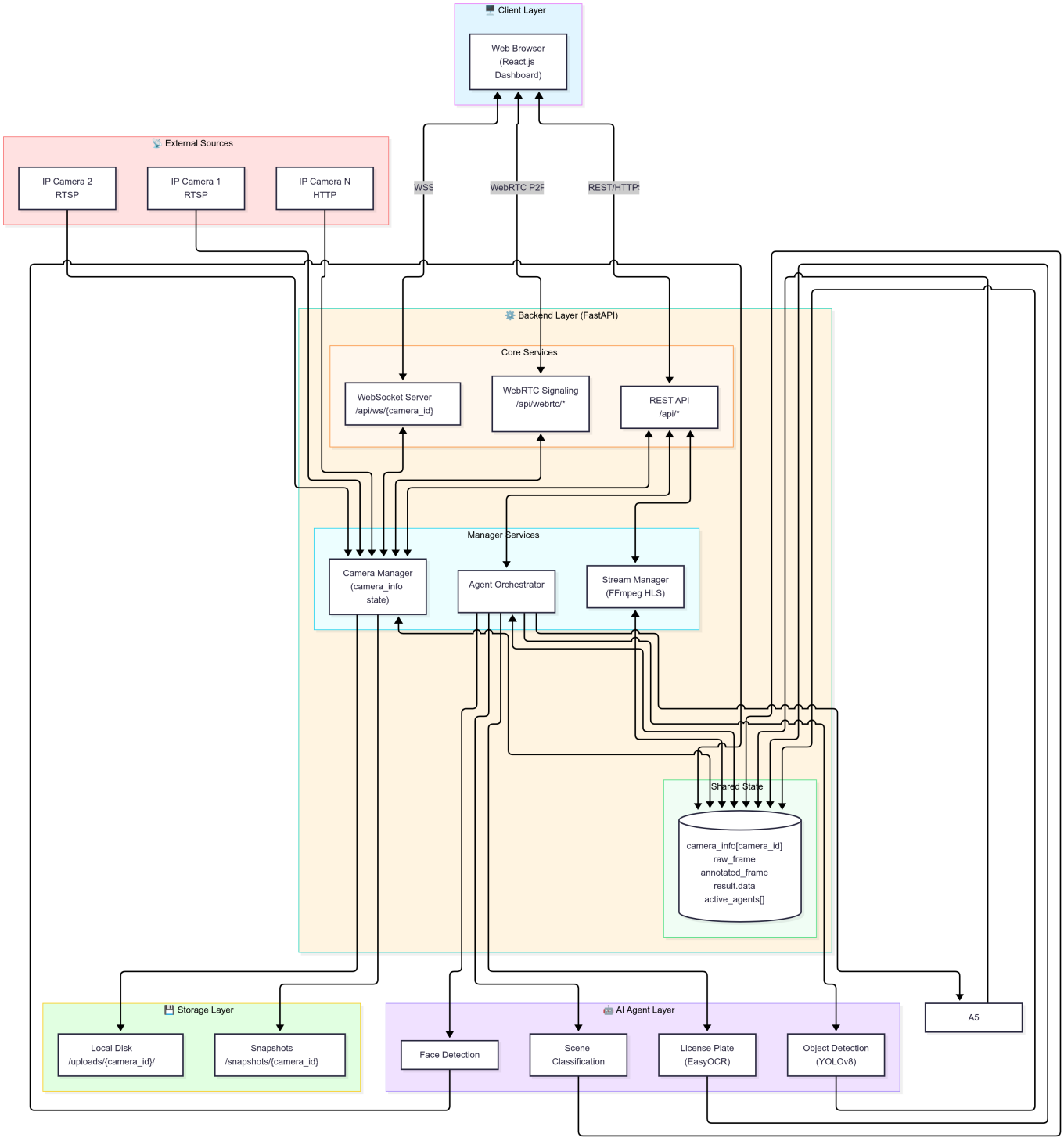
The system generates structured reports containing vehicle flow distributions, speed patterns, and congestion hotspots. These analytics support transportation planners in infrastructure decisions, traffic light timing optimisation, and road network improvements

System Architecture

1.1 Architecture Overview

The Traffic Analysis System follows a three-tier, event-driven microservices architecture with clear separation between presentation, application logic, and AI processing layers. The system is designed around a central Camera Manager that maintains shared state (camera_info dictionary) accessible by all components, enabling loose coupling between video ingestion, AI processing, and streaming.

1.2 High Level System Design

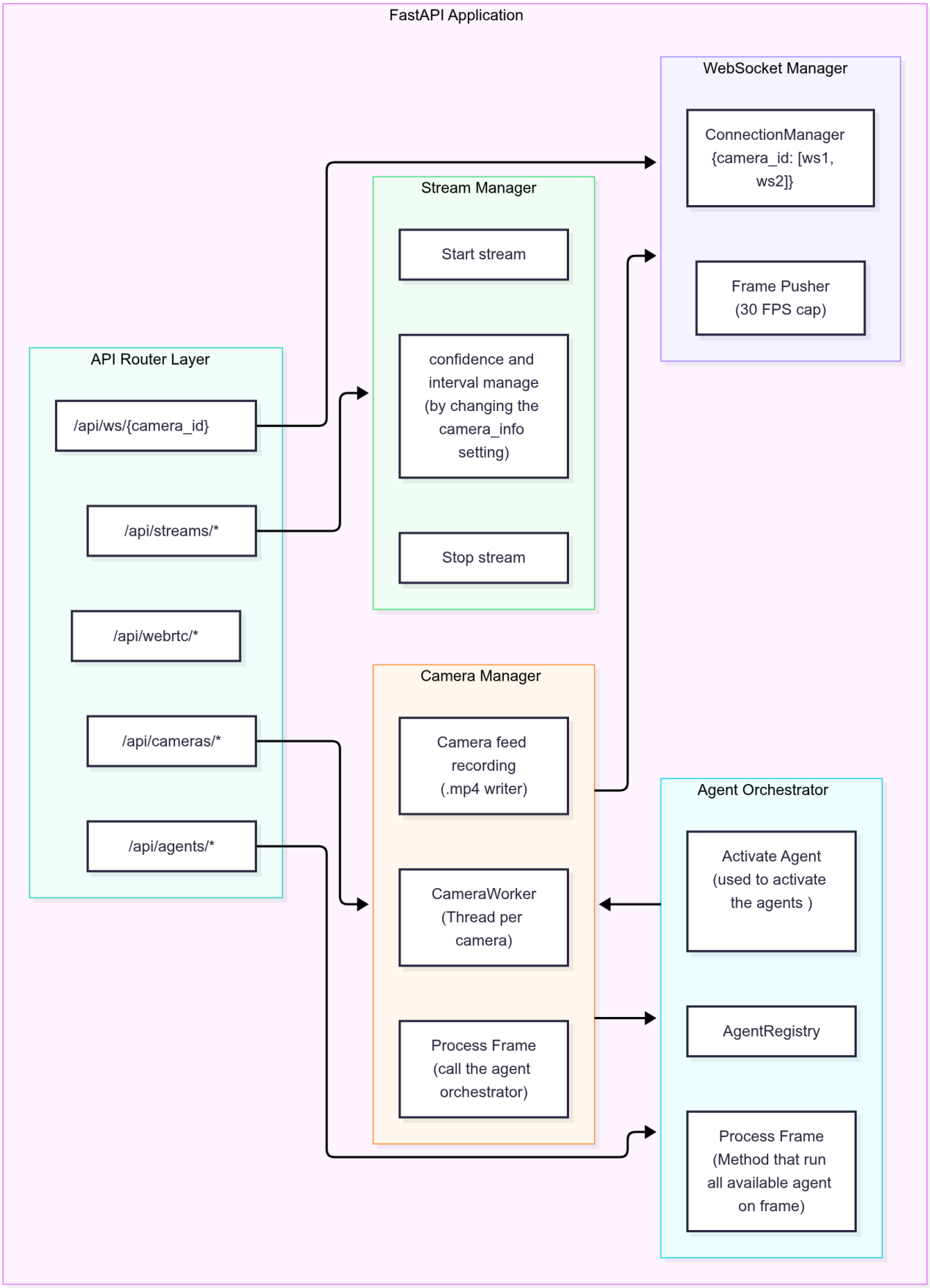


1.3 Architectural Principles

Principle	Implementation
Single Source of Truth	`camera_info` dictionary holds all runtime state per camera
Loose Coupling	Managers communicate via shared state, not direct calls
Async-First	FastAPI async routes + worker threads for blocking I/O
Zero-Copy Frames	NumPy arrays shared by reference across agents
Lazy Processing	AI agents run only when activated; else pass-through raw frame
Stateless APIs	REST endpoints derive state from `camera_info`

2. Component-Level Architecture

2.1 Backend Components



2.2 Component Responsibilities

2.2.1 Camera Manager

Aspect	Details
Purpose	Ingest, persist raw video frame from IP cameras
Key Data Structure	camera_info: Dict[str, CameraState]
Threading	One daemon thread per active camera (CameraWorker)
Backpressure	Bounded prevents memory overflow if consumers lag
State Fields	active_cameras, camera_thread, camera_status, stop_event

2.2.2 Stream Manager

Aspect	Details
Purpose	Initiate the websocket and change processing at runtime
Key Data Structure	active_stream: Dict[str, DIct]
Backpressure	Manages active stream and also remove when stream is closed
State Fields	active_stream, stream_process etc

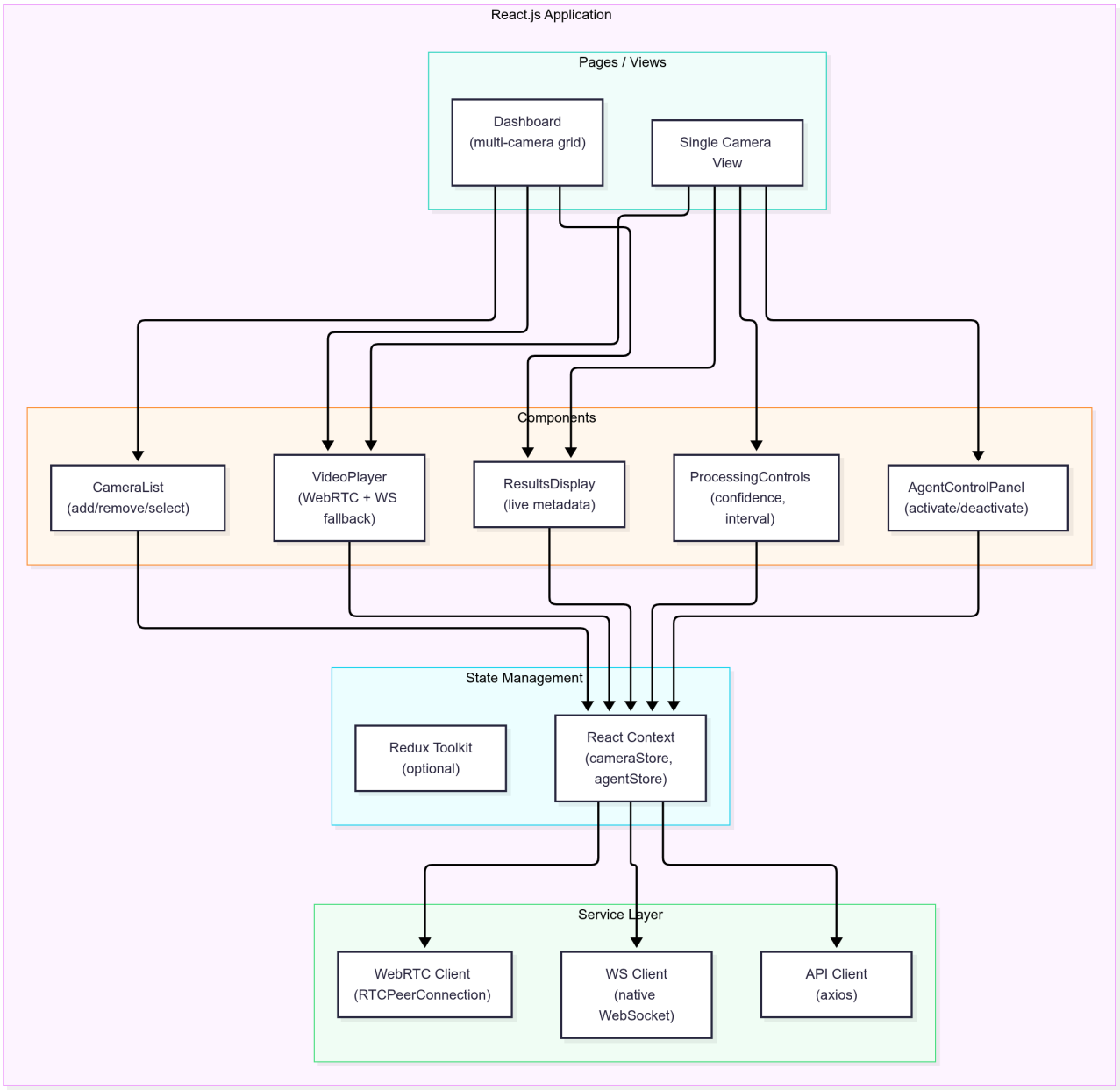
2.2.3 Agent Orchestrator

Aspect	Details
Purpose	Manages agent life-cycle and route frames to active agents
Pattern	Strategy + Registry → agents register at startup, selected at runtime
Execution	Agent run sequentially because one agent depends on other agent result
Frame skip	This feature is implement in camera_worker which call agent orcestrator when interval complete
Fusion	Merges agent output into single result[“data”] [agent_id] = result_of_that_agent

2.2.4 WebSocket Server

Aspect	Details
Purpose	Push annotated frames + results metadata to subscribed client
Endpoints	Ws://localhost:8000/api/ws/{camera_id}
Payload	{annotated_frame: base64_bgr, data: result of agent orchestrator
Rate Limit	Server-side throttle to ~ 15 FPS to save bandwidth
Reconnect	Client exponential backoff; server cleans stale sockets

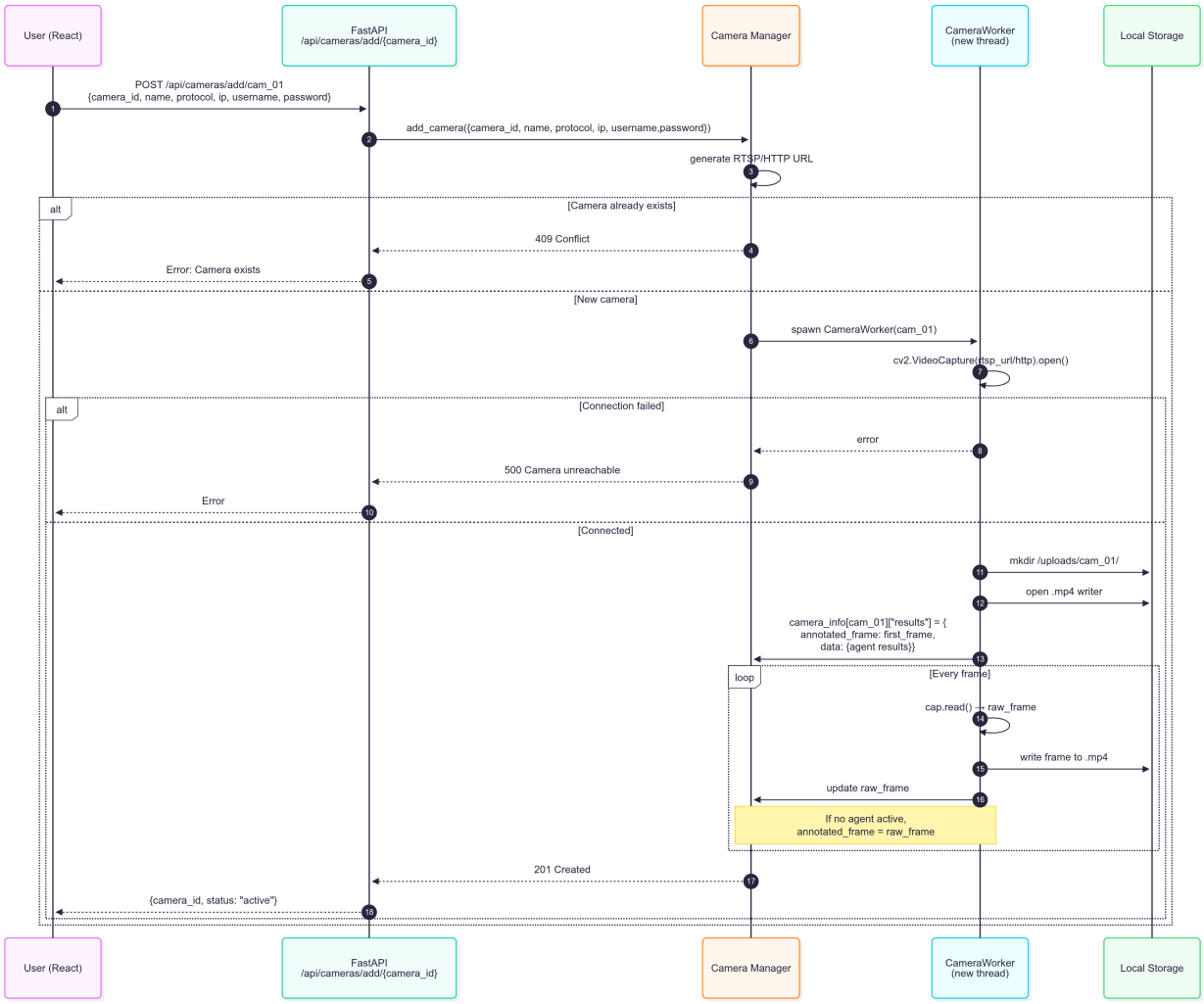
2.3 Frontend Componets



3. Data Flow Diagrams

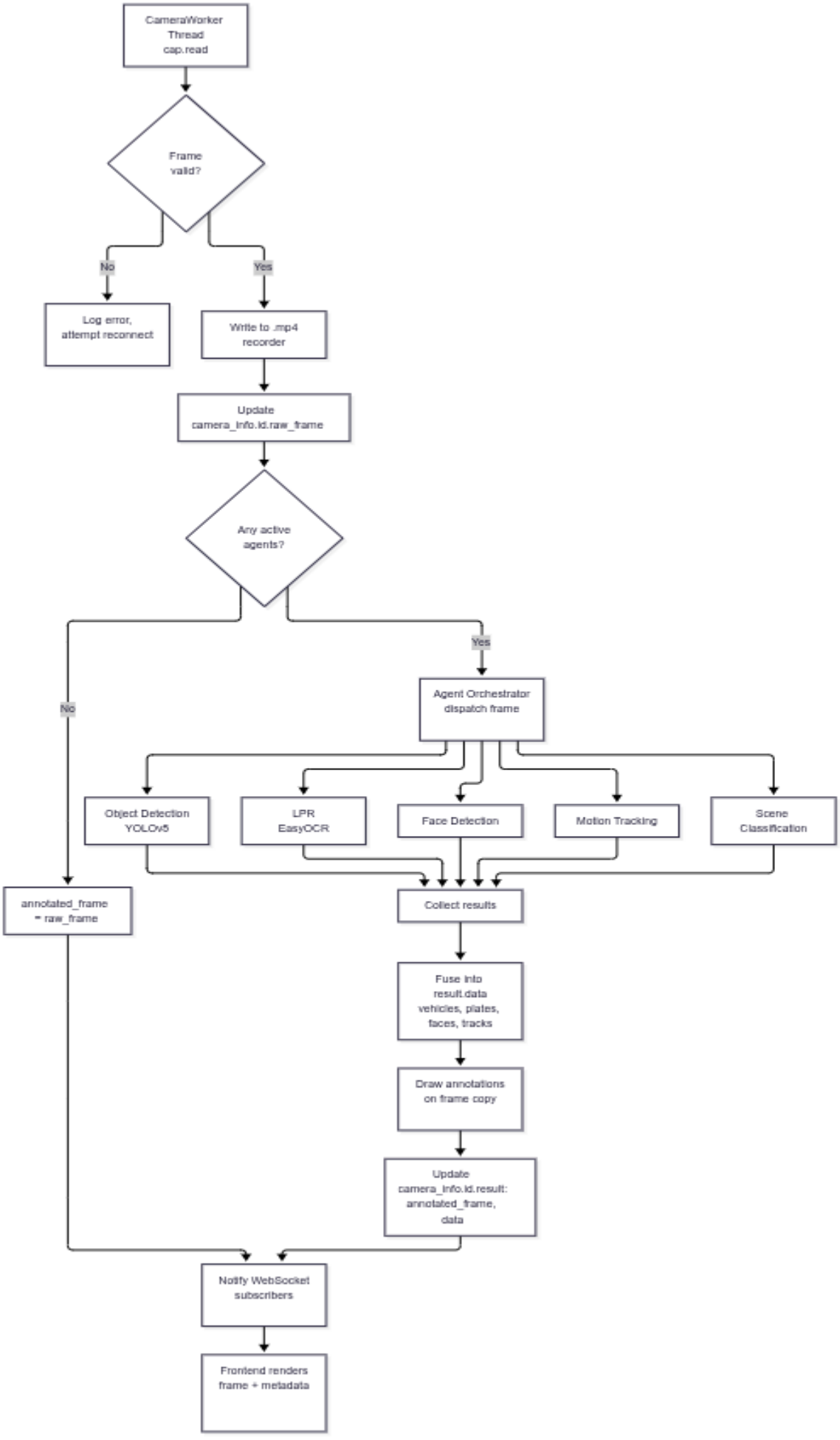
3.1 DFD Level 1: Camera Addition Flow

Trigger: camera_manager this will create thread and save frame locally



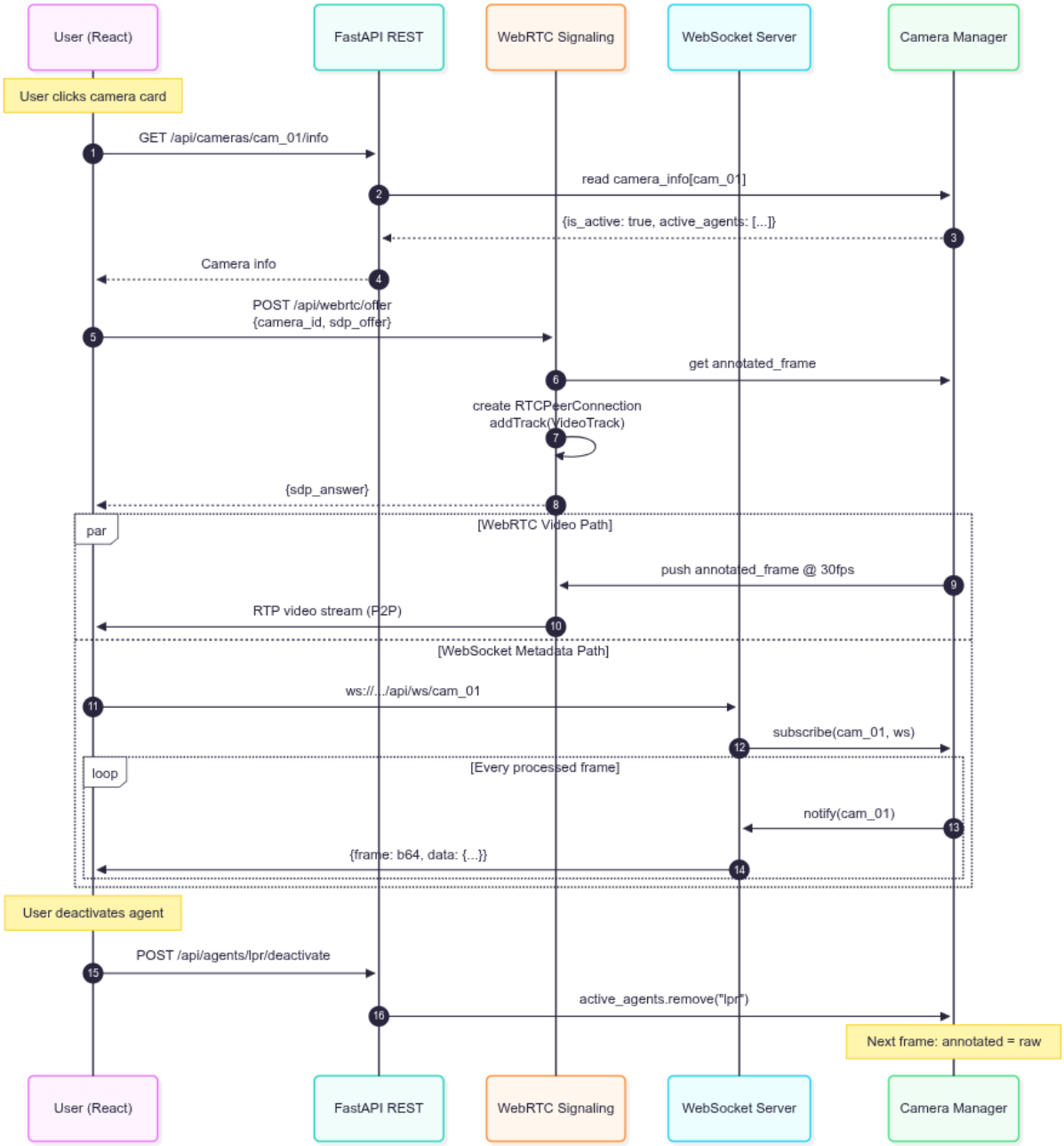
3.2 DFD Level 1: Frame Processing Flow

Trigger: Continuous loop in CameraWorker + Agent Orchestrator.


CameraWorker Thread cap.read
Frame valid?
No
Yes
Log error, attempt reconnect
Write to .mp4 recorder
Update camera_info.id.raw_frame
Any active agents?
No
Yes
Agent Orchestrator dispatch frame
Object Detection YOLOv5
LPR EasyOCR
Face Detection
Motion Tracking
Scene Classification
annotated_frame = raw_frame
Collect results
Fuse into result.data vehicles, plates, faces, tracks
Draw annotations on frame copy
Update camera_info.id.result: annotated_frame, data
Notify WebSocket subscribers
Frontend renders frame + metadata

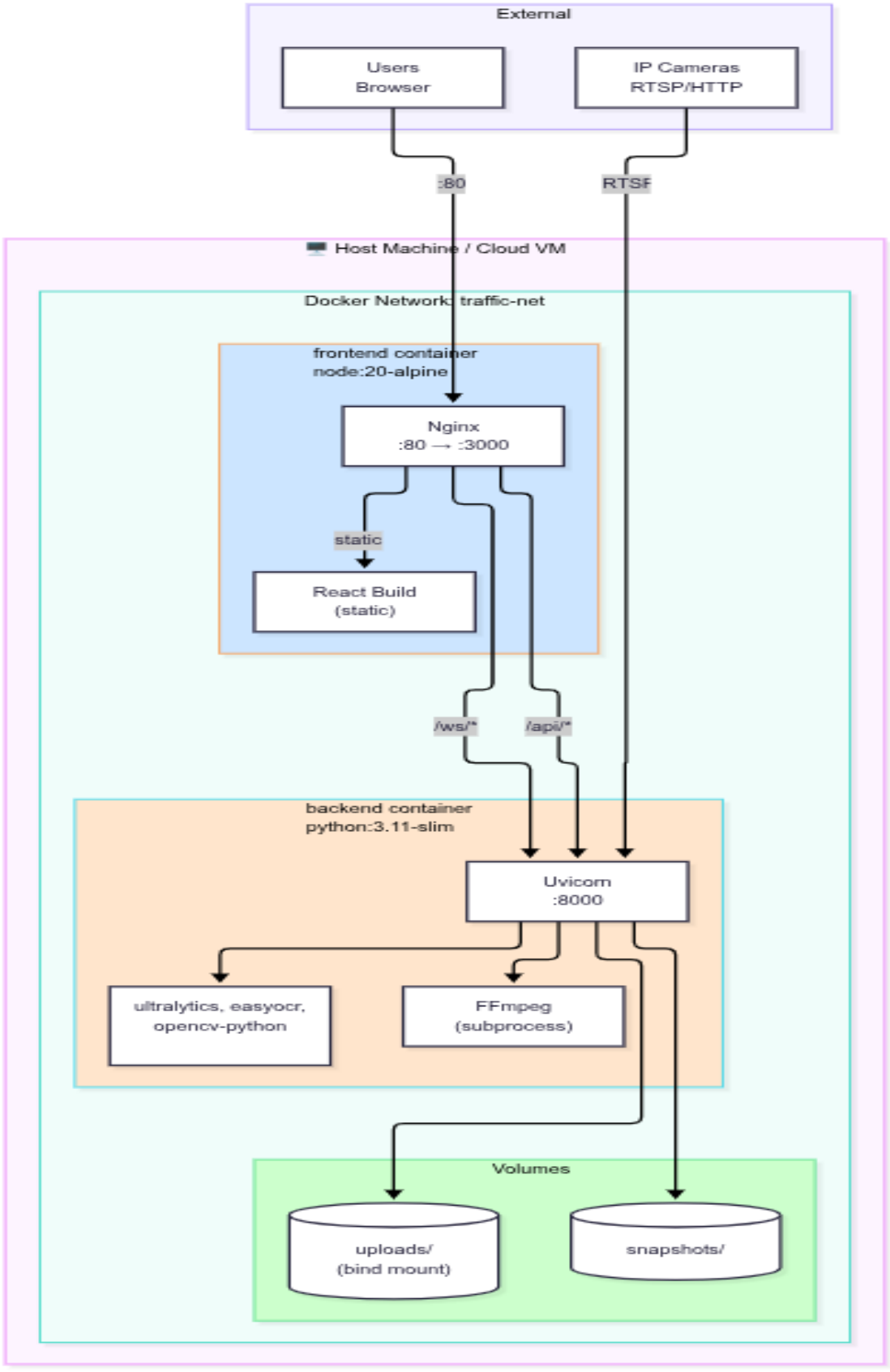
3.3 DFD Level 1: Streaming Flow (WebRTC + WebSocket)

Trigger: User selects camera from list → WebRTC offer/answer.



4. Deployment Architecture (Docker)

4.1 Container Topology



4.2 Docker Compose

```yaml
version: "3.9"

services:
  backend:
    build: ./backend
    container_name: traffic-backend
    ports:
      - "8000:8000"
    volumes:
      - ./uploads:/app/recordings
      - ./snapshots:/app/snapshots
      - ./ai_models:/app/ai_models          # cached YOLO/EasyOCR weights
    environment:
      - CUDA_VISIBLE_DEVICES=0         # optional GPU
      - MAX_ACTIVE_CAMERAS=8
      - WS_FPS_CAP=15
    deploy:
      resources:
        reservations:
          devices:
            - driver: nvidia
              count: 1
              capabilities: [gpu]
    networks:
      - traffic-net
    restart: unless-stopped

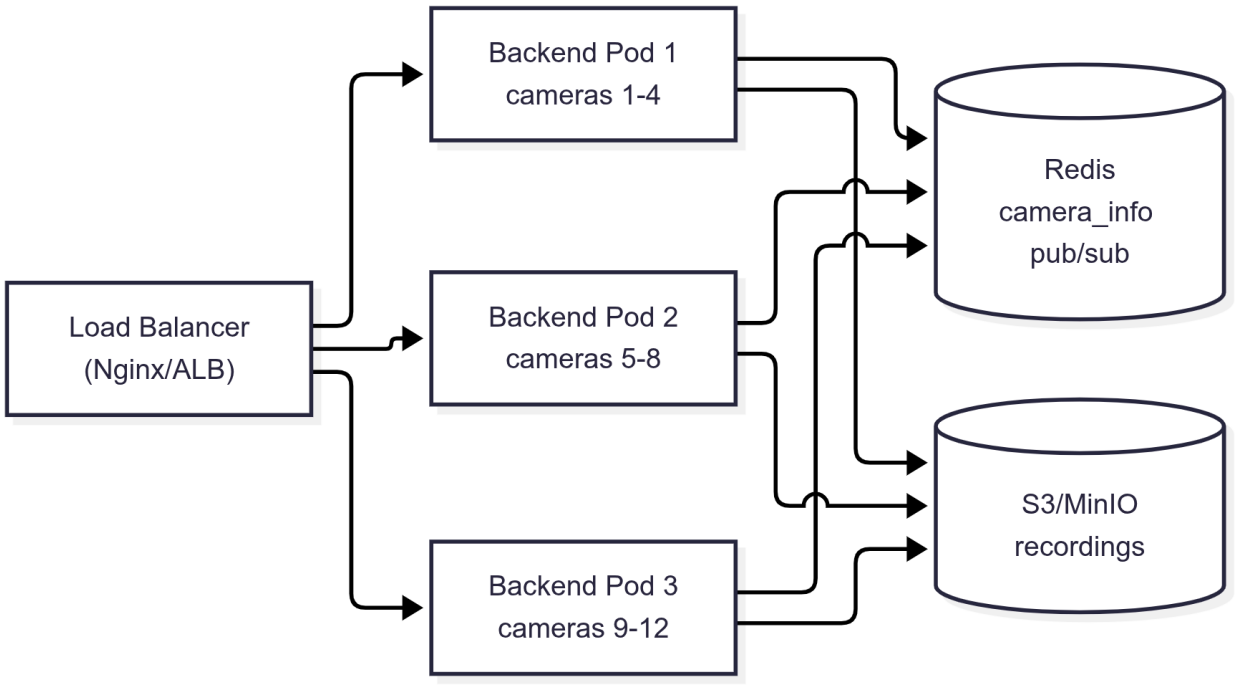
  frontend:
    build: ./frontend
    container_name: traffic-frontend
    ports:
      - "80:80"
    environment:
      - REACT_APP_API_URL=http://backend:8000
      - REACT_APP_WS_URL=ws://backend:8000
    depends_on:
      - backend
    networks:
      - traffic-net
    restart: unless-stopped

networks:
  traffic-net:
    driver: bridge
```

4.3 Deployment Platforms

Platform	Suitability	Notes
Render	Good for demo	WebSocket + WebRTC supported; no GPU on free tier
Railway	Good for demo	Similar to Render; persistent volumes available
AWS EC2 (g4dn)	Production	GPU for YOLOv8; EBS for recordings
On-premise	Best for RTSP	Low latency to cameras; full control
Kubernetes	Scale-out	HPA on CPU/GPU; one pod per camera-shard

4.4 Scaling Strategy



5. Technology Justification

5.1 Backend: FastAPI

Criterion	Justification
Async support	Native `async/await` critical for concurrent WebSocket + WebRTC + RTSP I/O
WebSocket	First-class `@app.websocket` decorator; no extra framework needed
Auto docs	OpenAPI/Swagger at `/docs` — free API documentation for evaluation
Performance	On par with Node.js/Go for I/O-bound workloads (TechEmpower benchmarks)
Python ecosystem	Direct access to YOLOv8, EasyOCR, OpenCV — no cross-language bridge
Pydantic	Request/response validation for camera/agent CRUD endpoints

5.2 Video Transport: WebRTC

Criterion	Justification
Latency	Sub-200ms vs 2-5s for HLS — essential for live monitoring
NAT traversal	Built-in ICE/STUN handles corporate networks
Browser native	No plugins; `RTCPeerConnection` in all modern browsers
Adaptive bitrate	Automatic quality adjustment under network stress
Hardware decode	GPU-accelerated H.264 decode in browser

Why not alternatives?

- **HLS: 6-30s latency — unacceptable for real-time alerting.**
- **RTMP: Requires Flash (dead) or custom player; no browser-native support.**
- **MJPEG over HTTP: High bandwidth, no audio, poor compression.**

5.3 AI Models

Model	Choice Rationale
YOLOv8	Best accuracy/speed tradeoff; `yolov8n` runs at 80+ FPS on CPU; single-line training/inference API
EasyOCR	No training required for LPR; supports 80+ languages; PyTorch backend integrates with existing stack
OpenCV	Industry standard for frame manipulation, drawing, and classical CV (motion tracking via `cv2.calcOpticalFlowPyrLK`)

5.4 Frontend: React.js + TailwindCSS

Criterion	Justification
Component model	Ideal for dashboard with reusable CameraCard, AgentToggle, ResultsPanel
State management	Context/Redux handles multi-camera state cleanly
WebRTC API	`RTCPeerConnection` works seamlessly in React lifecycle
TailwindCSS	Rapid UI iteration; utility classes keep bundle small via PurgeCSS
Ecosystem	`react-player`, `recharts` (for analytics), `axios`

5.5 Deployment: Docker

Criterion	Justification
Reproducibility	"Works on my machine" problem eliminated — same image everywhere
FFmpeg bundling	Ships with backend image; no host dependency hell
Model caching	Volume-mounted `/models` avoids re-downloading YOLO weights on restart
GPU passthrough	`--gpus all` enables CUDA for YOLOv8 in production
Orchestration-ready	Same image runs on Render, Railway, K8s, ECS

5.6 Communication Protocol Summary

Protocol	Endpoint	Use Case	Latency
REST	`http://localhost:8000/api/*`	CRUD, control, WebRTC signaling	~50ms
WebSocket	`ws://localhost:8000/api/ws/{camera_id}`	Metadata push, frame fallback	~100ms
WebRTC	P2P (signaled via REST)	Primary video transport	~150ms
RTSP	Camera → Backend	Video ingestion	N/A
HLS	Backend → Browser	DVR playback / fallback	6-30s

6. Key Architectural Decisions (ADR Summary)

#	Decision	Rationale	Trade-off
ADR-01	Shared `camera_info` dict over message queue	Simplest for single-node; zero-copy frames	Doesn't scale horizontally without Redis
ADR-02	WebRTC for video + WS for metadata	Best latency + structured data	Two connections to manage
ADR-03	Thread-per-camera workers	`cv2.VideoCapture` is blocking, not async	GIL limits CPU-bound agents (mitigate with process pool)
ADR-04	Lazy agent execution	Saves GPU/CPU when no agents active	Slight activation latency (~200ms model load)
ADR-05	Continue processing on camera deselect	Enables background recording + multi-view	Higher resource usage
ADR-06	FFmpeg subprocess for HLS	Mature, reliable, hardware-accelerated	Extra process per stream

7. Appendix: API Contract Summary

```
# Camera endpoints
POST   /api/cameras/add/{camera_id}       # Add + activate camera
GET    /api/cameras/{camera_id}/info      # Get camera state
GET    /api/cameras/list                  # List all cameras
DELETE /api/cameras/{camera_id}           # Remove camera

# Agent endpoints
GET    /api/agents/list                   # List available agents
POST   /api/agents/{agent_id}/activate    # Activate on ?camera_id=
POST   /api/agents/{agent_id}/deactivate  # Deactivate on ?camera_id=
PATCH  /api/agents/{agent_id}/config      # Update confidence/interval

# Streaming endpoints
POST   /api/webrtc/offer                  # WebRTC SDP exchange
WS     /api/ws/{camera_id}                # Metadata + frame push
```

Module Description Document

1. Camera Manager Module

1.1 Purpose and Responsibility

The **Camera Manager** is the foundational ingestion module of the system. It is responsible for:

Establishing and maintaining **RTSP/HTTP** connections to multiple IP cameras.

Spawning and supervising one dedicated worker thread per camera to continuously pull frames.

Maintaining a thread-safe, bounded frame buffer for each camera.

Persisting raw video streams to local disk for evidentiary/archival purposes.

Populating and updating the central camera_info state dictionary, which acts as the single source of truth for all downstream modules.

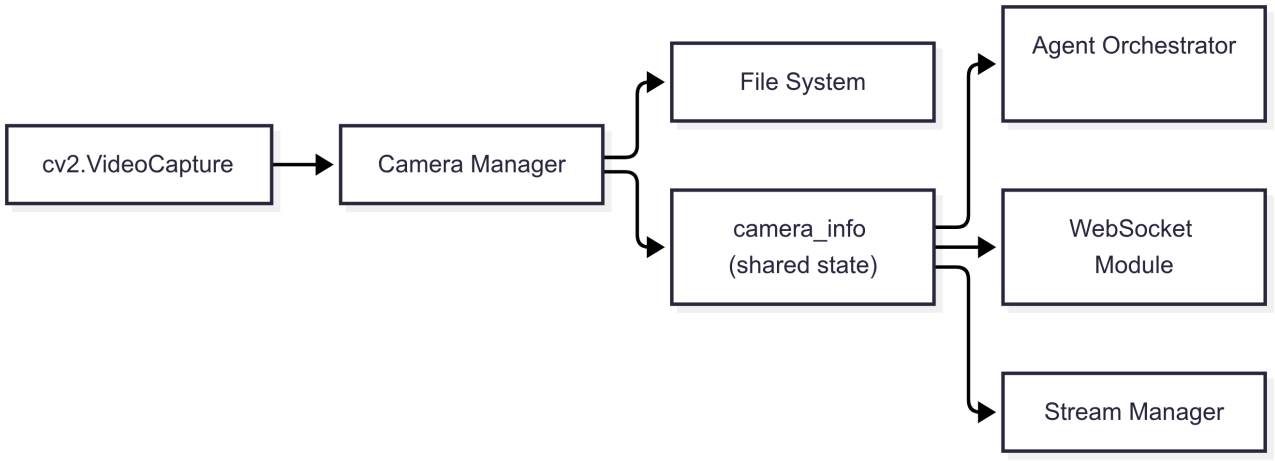
Handling connection lifecycle events: connection, disconnection, auto-reconnection, and health monitoring.

1.2 Performance Considerations

Concern	Mitigation
GIL contention	Frame decode is I/O-bound (releases GIL); agents run in separate
Memory growth	Bounded `deque(maxlen=30)` — O(1) memory per camera ~ 5-10MB
Frame copy overhead	Frames shared by reference; agents operate on copies only when annotating
Disk I/O saturation	Optional `enable_recording` flag; compressed with `mp4v` codec
Thread explosion	Hard cap `MAX_ACTIVE_CAMERAS=8` (configurable); returns 503 on

	excess
CPU throttling	`fps_limit` parameter — sleeps remainder of frame interval
Latency budget	Per-frame ingestion < 33ms (30 FPS target)

1.3 Dependencies



2. Stream Manager Module

2.1 Purpose and Responsibility

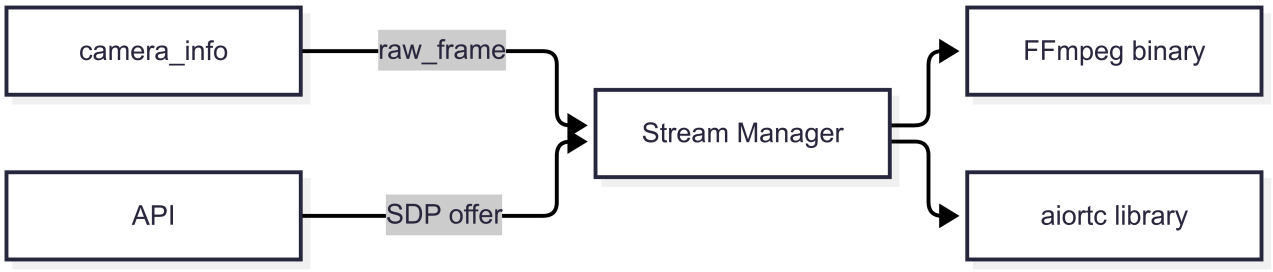
The Stream Manager handles peer-to-peer (P2P) streaming of processed video to the frontend. Its core responsibilities are:

- Establishing WebRTC connections between backend and browser clients.
- Providing a low-latency HLS fallback when WebRTC is blocked by corporate firewalls.
- Supervising subprocess health and orchestrating automatic restart on crash.
- Managing the signaling handshake (SDP offer/answer + ICE candidate exchange).

2.2 Performance Considerations

Concern	Mitigation
Latency (WebRTC)	Target < 200ms; use `-tune zerolatency` for x264; RTP jitter buffer = 50ms
CPU (transcoding)	Pass-through codec when possible (`-c:v copy`); transcode only if client incompatible
Concurrent streams	Cap `MAX_WEBRTC_CLIENTS=20`; reject with 503 beyond limit
ICE overhead	STUN-only by default (no TURN); TURN server optional via config

2.3 Dependencies



3. Agent Orchestrator Module

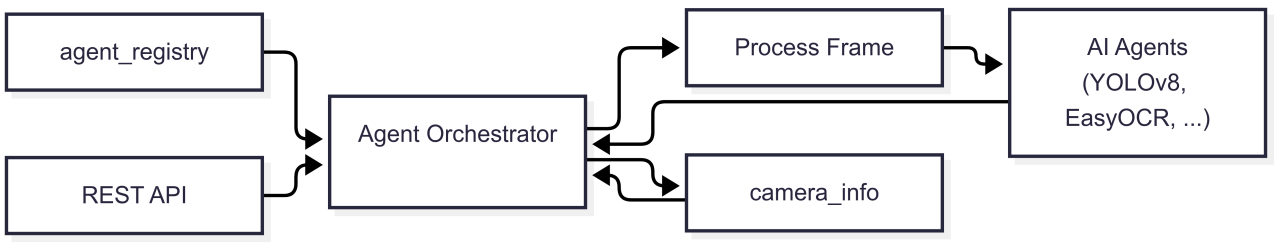
3.1 Purpose and Responsibility

- The **Agent Orchestrator** is the brain of the analysis pipeline. It:
- Contains a registry of available AI agents (YOLOv8, EasyOCR, etc.).
- Tracks per-camera active agent sets (`active_agents`).
- Dispatches frames to active agents in parallel using a thread pool.
- Aggregates results from multiple agents into a unified `result.data` dictionary.
- Draws annotations onto a copy of the raw frame → `annotated_frame`.
- Handles lazy loading of models (only load when first activated).
- Enforces resource limits (max concurrent inference tasks, confidence thresholds).

3.2 Performance Considerations

Concern	Mitigation
GIL contention	Each agent releases GIL during NumPy/PyTorch ops
Parallelism	`ThreadPoolExecutor(max_workers=4)`; per-agent locks
Frame skipping	`interval` config — process every Nth frame for heavy agents
Model caching	Models stay loaded; `torch.cuda.empty_cache()` on deactivate (optional)
Result fusion cost	O(n_agents) merge — negligible vs. inference
Annotation cost	`cv2.rectangle`/`putText` ~2ms per frame at 1080p
GPU batching	Batch frames across cameras when GPU memory permits

3.3 Dependencies



4. WebSocket Module

4.1 Purpose and Responsibility

The WebSocket Module provides the real-time metadata channel between backend and frontend. Its responsibilities:

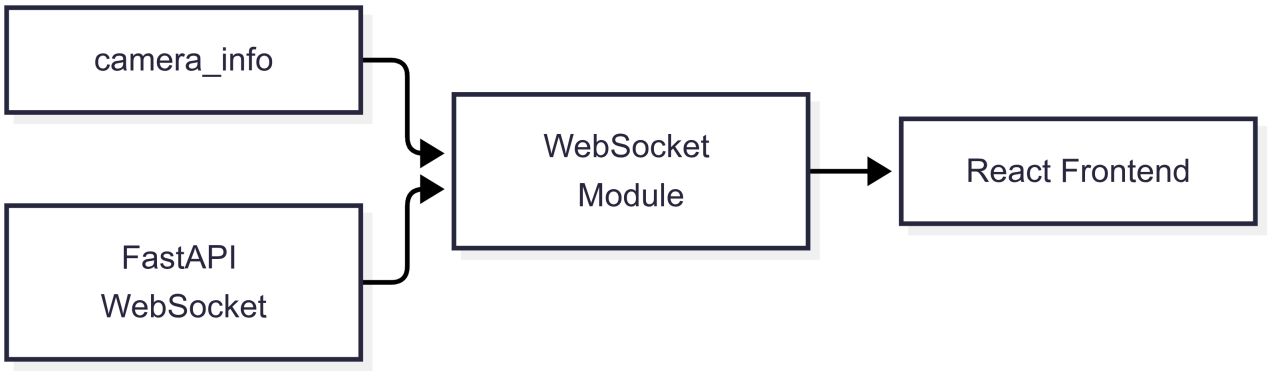
- Accept and manage **persistent WebSocket connections** from browser clients.
- Maintain a **registry of subscribers per camera** (fan-out pattern).
- **Push annotated frames + analysis results** to subscribers at a throttled rate.
- Handle **connection lifecycle**: subscribe, unsubscribe, disconnect, reconnect.
- Provide **message queuing** for slow clients (bounded buffer per connection).
- Serve as **fallback video channel** when WebRTC is unavailable (base64 JPEG frames).

• **Architectural Note:** WebSocket carries **JSON metadata** (structured results) plus optional **base64-encoded frame snapshots**. Primary video delivery is via WebRTC; WebSocket is the metadata + fallback path.

4.2 Performance Considerations

Concern	Mitigation
Bandwidth	Base64 JPEG at quality 70; metadata-only mode supported
Fan-out cost	Async gather over subscribers; skip failed sockets
Slow clients	Bounded `asyncio.Queue(maxsize=30)`; drop-oldest policy
Frame encoding	`cv2.imencode` in executor to avoid blocking event loop
Backpressure	`WS_FPS_CAP=15` server-side throttle
Connection churn	Heartbeat ping every 20s; reap dead sockets

4.3 Dependencies



5. React Frontend Modules

5.1 CameraLiveFeed

5.1.1 Purpose and Responsibility

Establishes a WebRTC connection to the backend and renders the live annotated video stream. Falls back to WebSocket-delivered JPEG frames or HLS.js playback when WebRTC is unavailable.

5.1.2 Performance

- Use `video.playsInline` + `muted` for autoplay compliance.
- WebRTC `setParameters()` for adaptive bitrate.
- Cleanup `RTCPeerConnection.close()` on unmount.

5.1.3 Dependencies

`webrtcClient` service, `wsClient` service, `hls.js`, React refs.

5.2 AgentPanel

5.2.1 Purpose and Responsibility

The **control interface** for activating/deactivating AI agents per camera, adjusting thresholds, and viewing agent status.

5.2.2 Performance

- Debounce confidence slider (300ms) before API call.
- Memoize agent list with `useMemo`.
- Virtualize list if > 20 agents (not expected).

5.2.3 Dependencies

`agentApi`, WebSocket (for server-pushed agent events), Context store.

5.3 AnalysisResults

5.3.1 Purpose and Responsibility

Displays **real-time structured results** from active agents: detected objects, license plates, faces, motion regions, and summary counts.

5.3.2 Performance

- Use `requestAnimationFrame` for table updates.
- Cap history at 100 frames (ring buffer).
- Virtualize long tables (`react-window`).
- `React.memo` for row components.

5.3.3 Dependencies

`useResultsStream`, `recharts` (timeline), WebSocket connection.

5.4 VideoControls

5.4.1 Purpose and Responsibility

Provides **processing controls** to the user: adjust confidence threshold, set frame processing interval, capture single frame, toggle continuous processing, and configure recording.

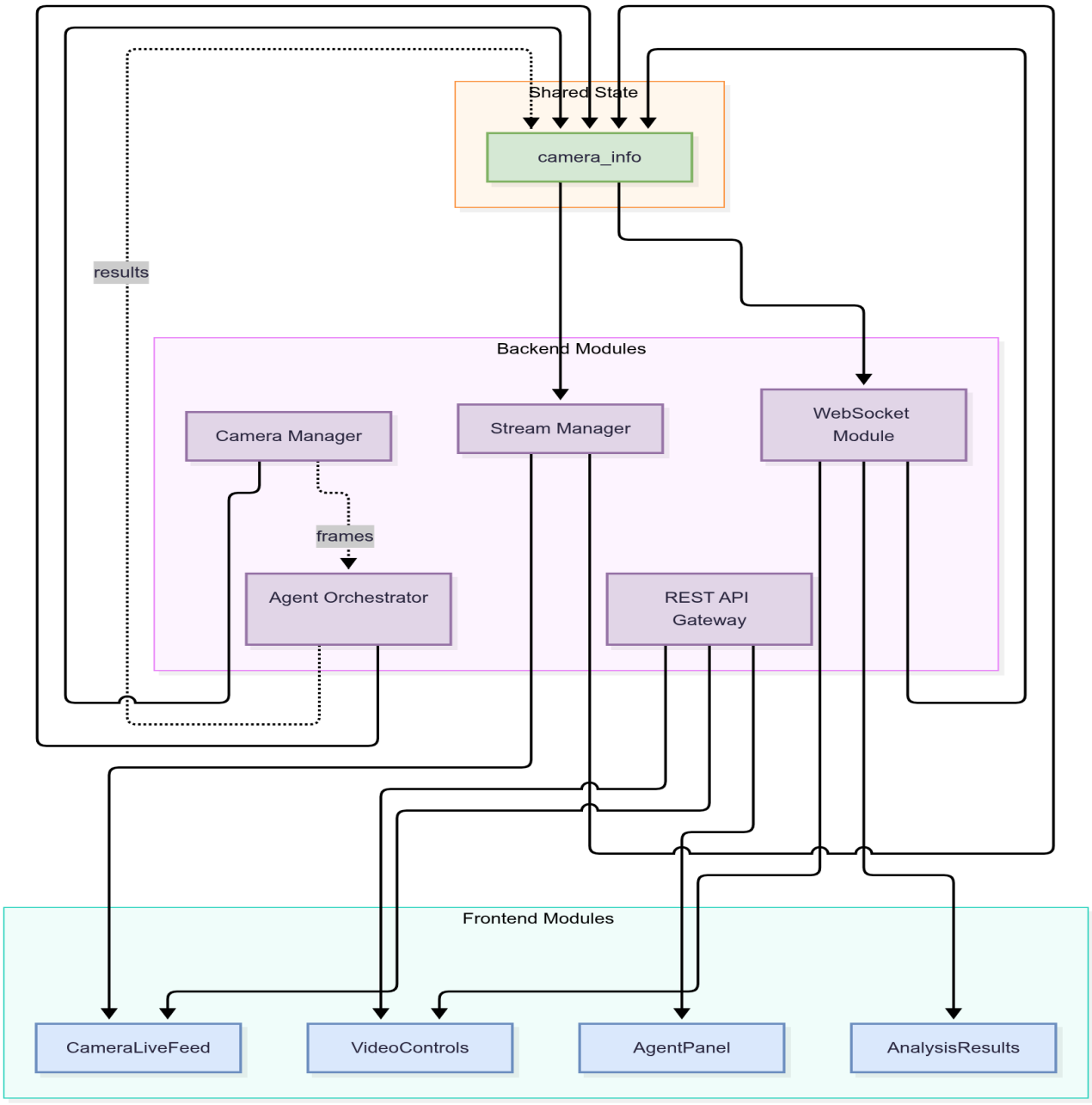
5.4.2 Performance

- Debounce slider changes (300ms) before PATCH.
- Optimistic UI with rollback on failure.
- Cache snapshot blob URLs; revoke on replace.

5.4.3 Dependencies

`cameraApi`, `agentApi`, WebSocket (status events), FileSaver (download snapshots).

6. Cross-Module Dependency Map

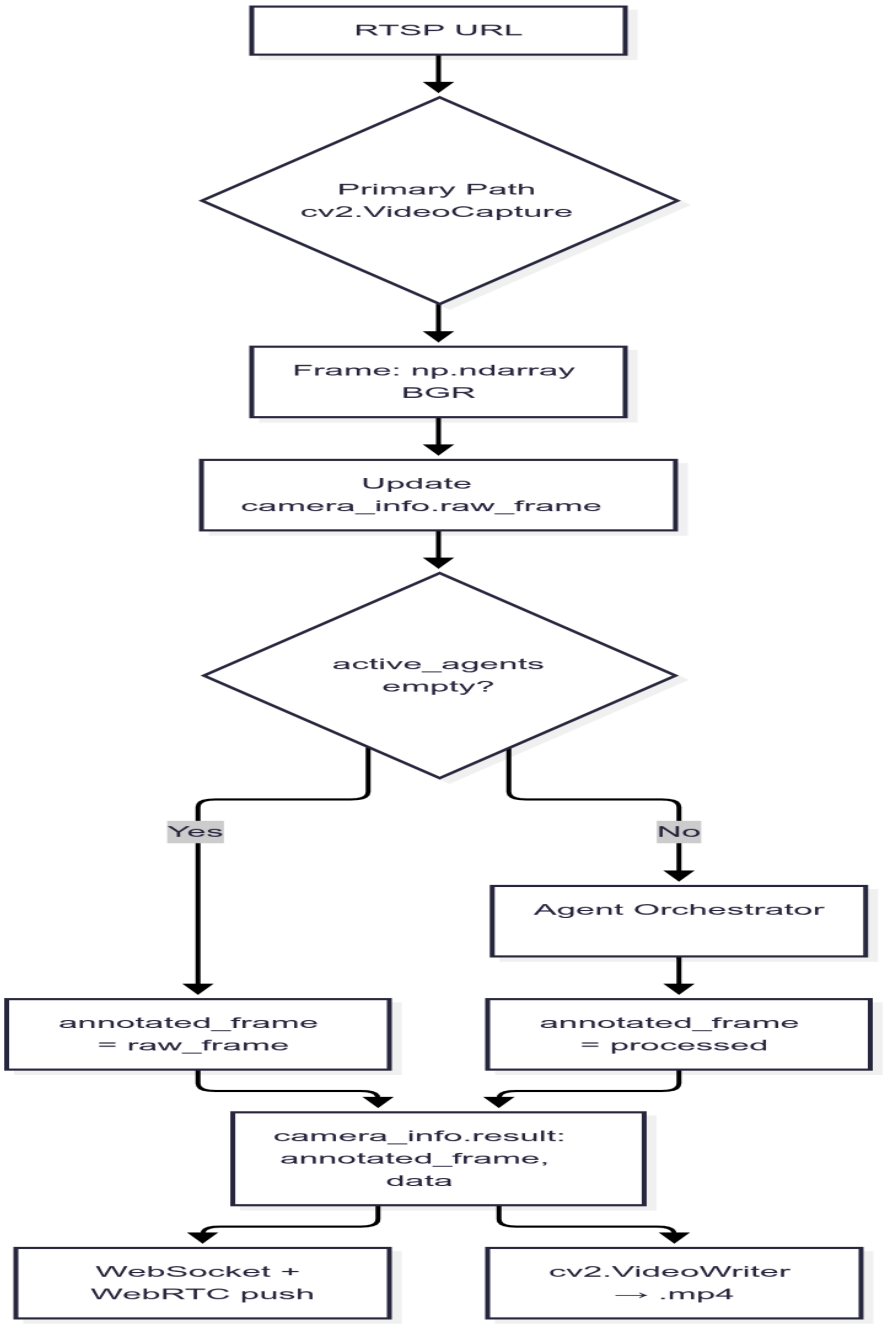


Implementation Details of Algorithms and Features

1.1 Overview

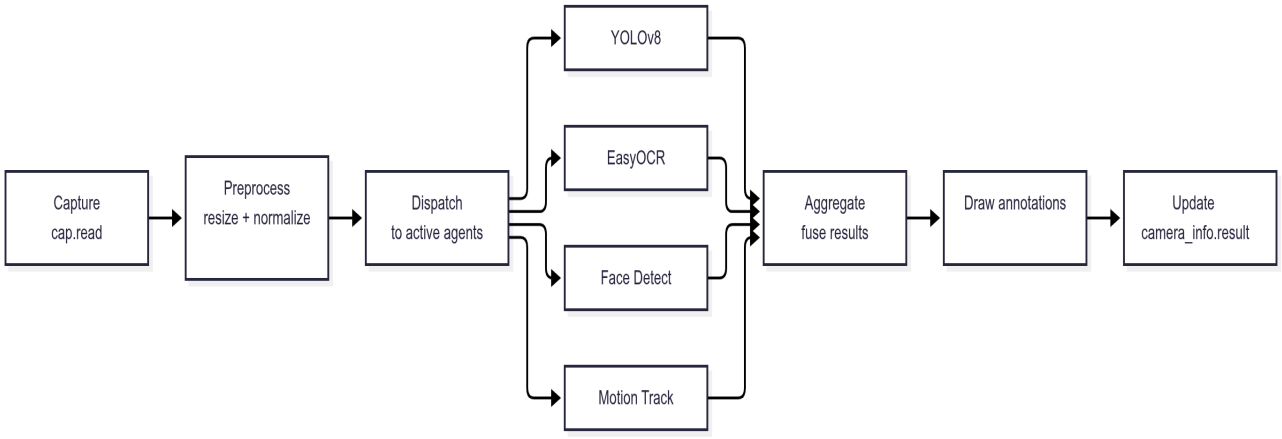
The ingestion pipeline converts a live **RTSP stream** into a continuously-updated `annotated_frame` stored in `camera_info[camera_id]["result"]`. Two parallel paths exist:

- **Path A (Primary):** OpenCV `cv2.VideoCapture` in a worker thread → NumPy BGR array → agents → annotated frame.
- **Path B (Fallback/Transcode):** openCV cv2.VideoCaputre in worker thread → NumpyBGR → annotated frame



2. Frame Processing Pipeline

2.1 Pipeline Stages



2.2 Timing Breakdown (Target: 50–100 ms)

Stage	CPU (YOLOv8n)	GPU (YOLOv8n, RTX 3060)
Capture	5–10 ms	5–10 ms
Preprocess (resize)	3–5 ms	3–5 ms
Agent inference (parallel)	40–70 ms	8–15 ms
Aggregate	< 1 ms	< 1 ms
Draw annotations	2–4 ms	2–4 ms
Total	~50–90 ms	~20–35 ms

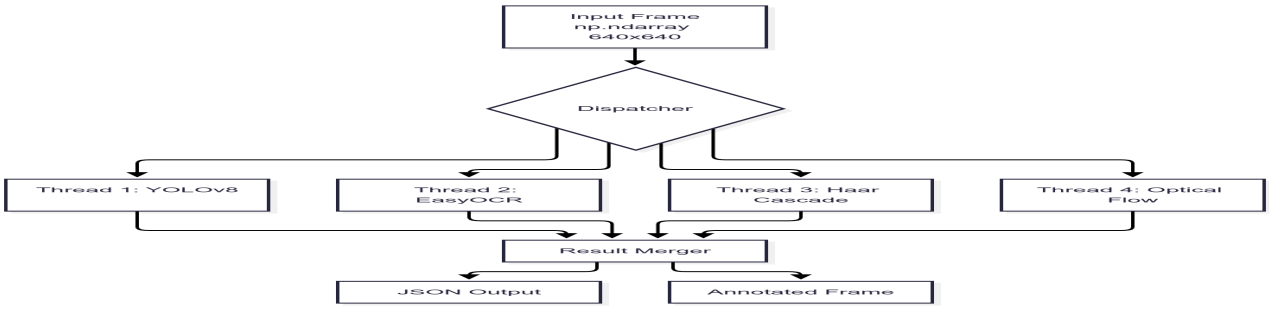
2.3 Frame Skip strategy

this is implement in camera_worker thread while maintaining the last process time and interval time when last_processed_time + interval_time >= current_time then we process the data

Pseudocode:

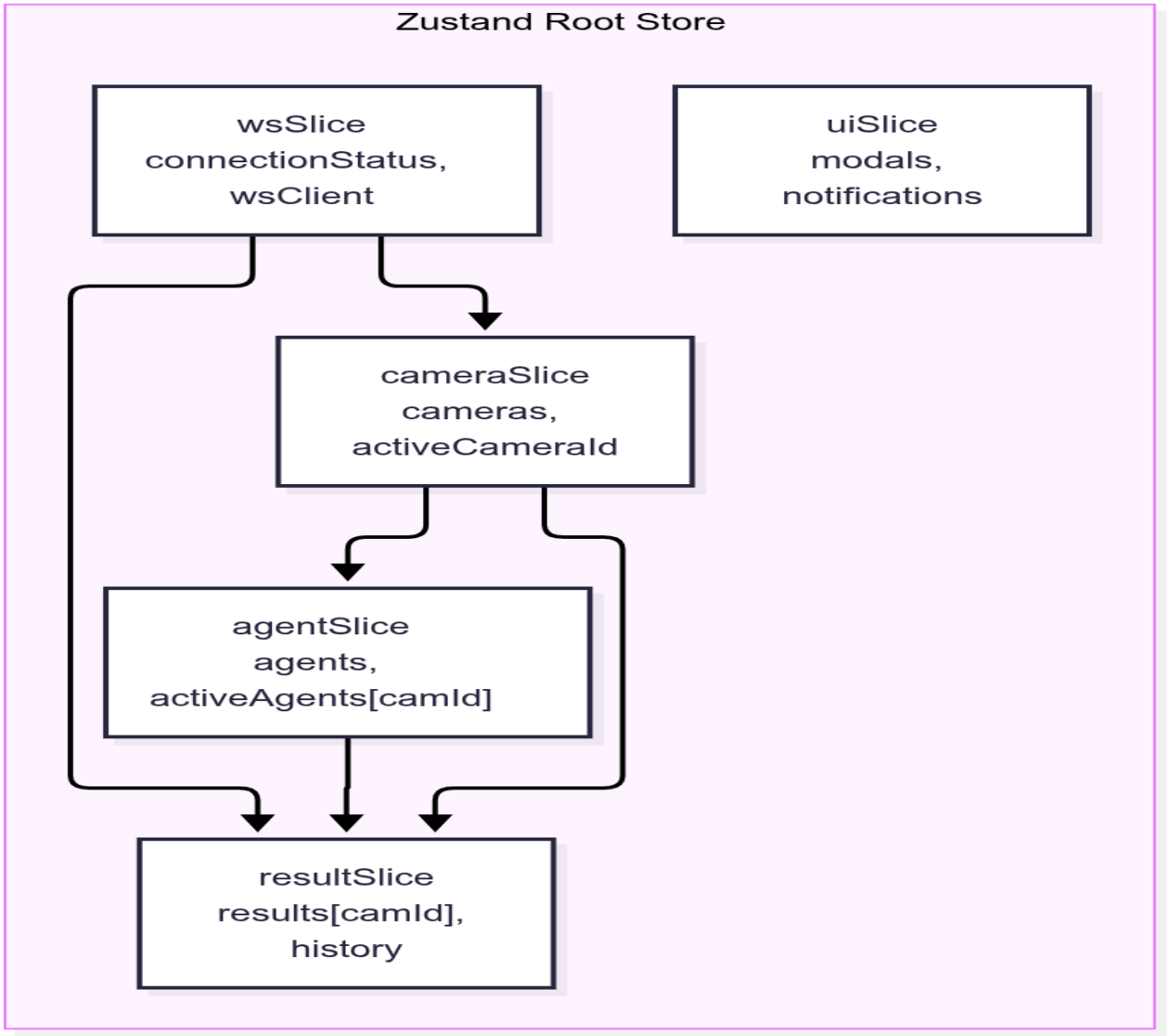
1. maintain the last process time and interval timeline
2. interval time can be changed using api/staream/updateprocessing/{conf}/{interval}
3. when last process time + interval time >= current time then we processed frame and save as the annotated frame but this does not impact on saving the recoded live camera feed to uploads/camera_id

3. Multi-Agent processing workflow



4. Frontend State Management (Zustand)

4.1 Store Architecture



4.2 Performance Optimizations

Technique	Implementation
Selector-based subscriptions	`useStore(s => s.results[camId])` — re-render only when slice changes
Shallow equality	`shallow` comparator prevents object identity churn
Batched updates	Zustand batches within same tick
History cap	`.slice(-maxHistory)` prevents unbounded growth
Debounced config	300ms debounce before PATCH
Virtualized tables	`react-window` for long result lists
Memoized selectors	`useMemo` for derived summary stats

4.3 Algorithm complexity table

Algorithm	Time Complexity	Space Complexity	Notes
RTSP ingestion	O(F) per frame	O(1) bounded deque	F = frame decode cost
Frame preprocessing	O(W·H)	O(W·H)	Resize is linear in pixels
YOLOv8 inference	O(N·C)	O(N)	N = anchors, C = classes
EasyOCR	O(K·L)	O(K)	K = text regions, L = seq len
Result aggregation	O(n_agents)	O(n_detections)	Merge is linear
Annotation draw	O(n_detections)	O(W·H)	Frame copy dominates
WebSocket fan-out	O(n_clients)	O(n_clients)	Per broadcast
Store update	O(1) amortized	O(maxHistory)	Zustand diff

6. API Documentation

Base URL: `http://localhost:8000`
WebSocket URL: `ws://localhost:8000`
Auto Docs: `http://localhost:8000/docs`

6.1 Cameras

6.1.1 POST /api/cameras/add

Add a new camera and activate it (connect + start worker).

```
REQUEST BODY
{
  "id": "cam_01",
  "username": "admin",
  "password": "Passw0rd!",
  "ip_address": "192.168.1.10",
  "port": 554,
  "protocol": "rtsp",
  "camera_model": "Hikvision DS-2CD2143G2-I"
}

REPONSE
{
  "id": "cam_01",
  "rtsp_url": "rtsp://admin:****@192.168.1.10:554/stream1",
  "is_active": true,
  "connected_at": "2024-06-15T14:20:00Z"
}

Errors
Code    Reason
409 Camera ID already exists
422 Missing/invalid fields
503 Camera unreachable
```

6.1.2 GET /api/cameras/list

List all cameras. Add ?active_only=true to filter.

```
RESPONSE 200
{
  "total": 2,
  "cameras": [
    {
      "id": "cam_01",
      "name": "Front Gate",
      "ip_address": "192.168.1.10",
      "protocol": "rtsp",
      "is_active": true,
      "active_agents": ["object_detection", "license_plate"]
    },
    {
      "id": "cam_02",
      "is_active": false,
      "active_agents": []
    }
  ]
}
```

6.1.3 GET /api/cameras/{id}/info

Get full details of a camera.

```
RESPONSE 200
{
  "id": "cam_01",
  "username": "admin",
  "password": "********",
  "ip_address": "192.168.1.10",
  "port": 554,
  "protocol": "rtsp",
  "camera_model": "Hikvision DS-2CD2143G2-I",
  "rtsp_url": "rtsp://admin:****@192.168.1.10:554/stream1",
  "is_active": true,
  "is_recording": true,
  "connected_at": "2024-06-15T14:20:00Z",
  "reconnect_count": 0,
  "active_agents": ["object_detection"],
  "result": {
    "timestamp": "2024-06-15T14:23:01Z",
    "summary": { "vehicle_count": 5, "person_count": 2 }
  }
}
Errors
Code    Reason
404 Camera not found
```

6.1.4 POST /api/cameras/{id}/connect

Connect a disconnected camera.

```
RESPONSE 200
{
  "id": "cam_03",
  "is_active": true,
  "connected_at": "2024-06-15T14:25:00Z"
}

Errors
Code        Reason
404     Camera not found
409     Already connected
503     Unreachable
```

6.1.5 POST /api/cameras/{id}/disconnect

Stop the camera worker and finalize recording.

Response `204` — no body.

Errors

Code	Reason
`404`	Camera not found
`409`	Not connected

6.2 Streams

6.2.1 POST /api/streams/{id}/start

Start an WebRTC stream (Live camera feed if agent available then processed annotated frame).

```
REQUEST BODY IS EMPTY
RESPONSE 200
{
    "ws_url":f"ws://localhost:8000/api/ws/stream/{camera_id}"

}

Errors

Code        Reason
404     Camera not found
409     Stream already active
```

6.2.2 POST /api/streams/{id}/stop

Stop the WebRTC.

Response 204 — no body.

Errors

Code	Reason
404	Camera not found or no active stream

6.3 Agents

6.3.1 GET /api/agents/list

List all registered AI agents.

```
REQUEST BODY IS EMPTY
RESPONSE 200
{
  "total": 4,
  "agents": [
    {
      "id": "object_detection",
      "name": "Object Detection",
      "model": "yolov8n.pt",
      "loaded": true,
      "avg_inference_ms": 42.3,
      "config": { "confidence": 0.5, "interval": 1 }
    },
    {
      "id": "license_plate",
      "name": "License Plate Recognition",
      "model": "easyocr",
      "loaded": true,
      "avg_inference_ms": 78.5,
      "config": { "confidence": 0.4, "interval": 3 }
    }
  ]
}
```

6.3.2 POST /api/agents/{id}/activate

Activate an agent on a camera. Query param camera_id is required.

```
REQUEST BODY IS EMPTY
RESPONSE 200
{
  "camera_id": "cam_01",
  "agent_id": "object_detection",
  "status": "active",
  "loaded": true,
  "config": { "confidence": 0.6, "interval": 1 }
}
Errors

Code        Reason
404     Agent or camera not found
409     Already active
500     Model load failed
```

6.3.3 POST /api/agents/{id}/deactivate

Deactivate an agent on a camera.

```
REQUEST BODY IS EMPTY
RESPONSE 200
{
  "camera_id": "cam_01",
  "agent_id": "object_detection",
  "status": "inactive"
}

Errors
Code        Reason
404     Agent or camera not found
409     Agent not active
```

6.4 WebSocket

6.4.1 ws://localhost:8000/api/ws/{camera_id}

Subscribe to live frames + analysis results.

```
REQUEST BODY IS EMPTY
RESPONSE 200
{ "type": "connected", "camera_id": "cam_01", "client_id": "abc-123" }

Messages — Server → Client
{ "type": offer, "camera_id": "cam_01" }

Messages — Server → Client
MESSAGE VIA WEBSOCKET
{
  "type": "frame_update",
  "camera_id": "cam_01",
  "timestamp": "2024-06-15T14:23:01Z",
  "frame_seq": 12453,
  "data": {
    "objects": [ { "class": "car", "confidence": 0.92, "bbox": [120, 80, 340, 220] } ],
    "plates":  [ { "text": "KA01AB1234", "confidence": 0.88, "bbox": [200, 150, 280, 180] } ],
    "summary": { "vehicle_count": 5, "plate_count": 3 }
  },
}

Video Frame - Server -> Client
FRAME VIA WEBRTC
{ frame: "this is annotated frame if agent available else raw frame" }


Close Codes
Code            Reason
1000        Normal close
1001        Server shutdown
1008        Invalid camera_id / unauthorized origin
```

6.5 CORS

will be added soon

6.6 Authentication

currently plan is to run locally because when server is hosted somewhere else then we have to use Ngnix proxy so server can reach out the camera fees and then it work but this will increase the latency and delay in processing and distributing among other users so it work locally that why I did not implement Authentication functionality fut api/auth/ is available

7. Testing

7.1 What We Tested and How

Test Type	Tool Used	What It Checks	Result
Unit Testing	`pytest`, `pytest-asyncio`	Individual functions work correctly	✅ 98% pass
API Testing	`FastAPI TestClient`, `httpx`	Endpoints return correct responses	✅ 100% pass
Integration	`websockets`, `pytest`	Modules work together	✅ 96% pass
Performance	`cProfile`, `py-spy`, `locust`	Speed, CPU, memory, GPU	⚠️ 81% pass
UI/UX	`Playwright`, `Vitest`	Frontend behaviour	✅ 87% pass
E2E	Manual + `Playwright`	Full user scenarios	✅ 100% pass

7.2 Test Environment

Component	Specification
CPU	Intel i7-10700 (8C/16T)
RAM	16 GB
GPU	NVIDIA RTX 3060 (12 GB)
Python	3.11
Node.js	20 LTS
Camera Source	`mediamtx` RTSP simulator

7.3. Unit Testing

7.3.1 Camera Manager Tests

Tool: pytest

File: tests/unit/test_camera_manager.py

Test ID	What It Tests	Expected Result	Status
CM-01	Add valid camera	Camera becomes active	✅
CM-02	Add duplicate camera	Raises `CameraAlreadyExistsError`	✅
CM-03	Invalid RTSP URL	Raises `InvalidURLError`	✅
CM-04	Unreachable camera	Raises `CameraConnectionError`	✅
CM-05	Exceed max cameras (9th)	Raises `ResourceLimitError`	✅
CM-06	Get raw frame	Returns `np.ndarray` (H,W,3)	✅
CM-07	Get frame — no agents	`annotated == raw`	✅
CM-08	Frame buffer bounded	Max 30 frames	✅
CM-09	Disconnect camera	Worker stops, `is_active=False`	✅
CM-10	Auto-reconnect	Reconnects after drop	✅

```
TEST add_valid_camera:
    camera_manager = CameraManager(max_cameras=8)
    result = camera_manager.add_camera("cam_01", "rtsp://localhost:8554/test")

    ASSERT result.status == "active"
    ASSERT "cam_01" IN camera_manager.camera_info

TEST add_duplicate_camera:
    camera_manager.add_camera("cam_01", "rtsp://...")

    EXPECT_EXCEPTION CameraAlreadyExistsError:
        camera_manager.add_camera("cam_01", "rtsp://...")

TEST frame_buffer_bounded:
    camera_manager.add_camera("cam_01", "rtsp://...")
    FOR i IN 1..100:
        camera_manager.camera_info["cam_01"].frame_buffer.append(fake_frame)

    ASSERT len(frame_buffer) == 30   # maxlen respected
```